Precision farming with deep learning and Tapaal model

Md Akram[1]

Contents

1 Introduction **2**

2 Deep Learning **4**

2.1 System Design 4

2.2 Dataset 4

2.3 CNN Structure 4

2.4 CNN implementation 5

2.5 Experimental Evaluation 7

2.6 Model Prediction Results 8

3 TAPAAL Model and Verification **9**

3.1 Use Case diagram 9

3.1.1 Use case diagram for Precision Farming 10

3.1.2 Use case diagram for Drones 11

3.1.3 Use case diagram for Autonomous Vehicles 12

3.1.4 Farmer Use Case 12

3.2 TAPAAL model for individual System 13

3.2.1 System_Controller Interface 13

3.2.2 Drones_And_Autonomous_Vehicles Interface with Time Variants 14

[1] md.akram@hshl.de

Verification and Validation of Cyber-Physical Systems

3.2.3 Farmer_Control Interface 15
3.3 TAPAAL Verification 15
3.3.1 System_Controller_Reachability 15
3.3.2 Farmer_Controller_Reachability 16
3.3.3 Deadlock 16
3.3.4 Controller_model_liveness 17

4 Competition **17**
4.1 Preparation for competition 18
4.2 Approach To Win the Competition 18

5 Conclusion **19**

6 Project Management with GitHub **19**

7 Declaration of Originality **19**

Abstract: Precision Farming is the combination of some modern technologies as a solution of some major problem in traditional Agriculture and which optimize agricultural productivity. This paper explores the uses of deep learning and model verification using TAPAAL in precision farming. The paper discusses deep learning approaches on the first while on the last about TAPAAL model and Verification process. A deep learning model trained using PyTorch and more than 5K images which able to identifies plant categories and weeds with high accuracy. Experimental data such as training and validation accuracy and losses help us to see the learning process as well as confusion matrices represent correct and wrong prediction. TAPAAL-verified model has three different components including System Controller, Drones and Autonomous Vehicles and Farmer Control able to work together to manage Precision Farming. There are total five specialized drones like monitoring, pesticide application, irrigation, harvest analysis and weed detection. Beside that two autonomous vehicles responsible for crop harvesting and weed removal. Meanwhile if any physical error or damage happen Farmer need to manually solve the error. This paper is a good illustration of using deep learning and formal verification methods in the field of precision farming.

1 Introduction

Agriculture is mandatory for maintaining ecosystems and providing base for human civilization. According with the increase of population growth, the demand of food is also increase. It is possible only by agriculture to manage food for billions of people. According the previous studies we can see agriculture has multiple problems like climate change,

Verification of Cyber-Physical Systems

irrigation, diseases, lack of agricultural land and capital as well as many more. Among them plant disease is the worst obstacle and challenge for agriculture production [Kr24]. For example, only for plant disease, between 20 and 40 percent of agricultural crop production is lost per year in the US [Ne21]. So if any how it is possible to identify the disease of crops in early stage then it is possible to use pesticide and fertilizer to control the disease which reduce production cost and increase crop production. Meanwhile any wrong prediction about plant disease will result uses of inappropriate fertilizers which may cause plant stress and nutritional deficiencies in the agricultural farm [AR21]. So, we can see identification of accurate plant, disease and weed is very important for using correct pesticide and fertilizer on time for better agricultural production. In such situation Precision Farming can be a good solution which is an agricultural method that monitors and controls targeted farming using data, technology, verification method and instruments including drones, sensors, and GPS to optimize productivity and sustainability. Precision Farming technology help the farmer to real-time monitoring, predictive analytics and automated tasks execution. Precision Farming can cover wide area within short time with high accuracy while traditional farming cannot cover such big area with short time and sometime prediction is not correct of farmer.

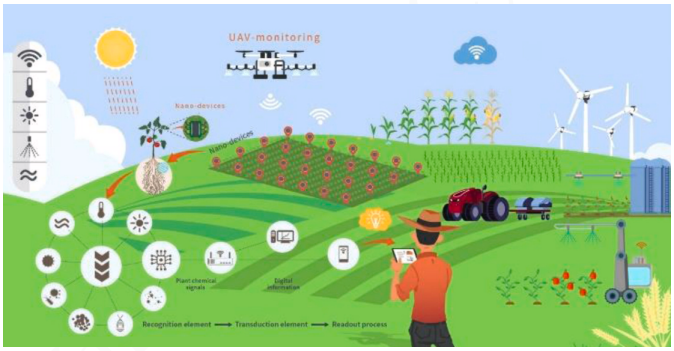


Fig. 1: Precision Farming [Za24]

Precision Farming use advanced methodologies like deep learning model developed in this study using PyTorch which facilitate identification of plant and weed categories with high accuracy based on photographic data. The model training process can be seen from the training and validation accuracy and losses curve while confusion matrix represent the true prediction and wrong prediction by the model.

Furthermore, this paper introduces a TAPAAL verified model where System Controller operating overall tasks among drones and autonomous vehicles. There are total five drones in five different rules including monitoring farm conditions, pesticide application, irrigation, harvest analysis and weed detection. System Controller get data from those drones then give instruction two autonomous vehicles for harvesting and weed removing from farm. System Controller ensures continues operations by verifying two system condition including system requirements and error identification process. Physical errors trigger the farmer control section for human intervention while network and system errors are addressed autonomously

through the error identification system. This paper represents the combination of deep learning with formal verification for Precision Farming which offers a scalable, efficient and sustainable solution in modern agriculture.

2 Deep Learning

Deep learning is a subset of machine learning that uses neural networks to automatically learn patterns from data. In the recent years we can notice the development of machine learning and several use of many important sector. Image processing method, a part of deep learning, widely uses in precision farming for plant, disease and weed identification. Convolutional Neural Networks (CNN) a most useful type of deep learning which has been the widely uses in the identification of plant categories and it's diseases. In deep learning section I did train a CNN model for identify several categories of plants, weed and parasite.

2.1 System Design

After analysis my requirements and user I did design a overall system which integrate deep learning for precision farming. We can see the architecture of this system on figure 2. There are total three primary component on the architecture including CNN modeling and training, Dataset and cloud storing and finally User interface layer. On the first layer I describe the CNN base Deep Learning model where I train that model with 5k images of 12 different categories of plant. On the second layer I describe about the cloud layer for storage trained model and dataset. At the end, on third layer I showed how to use the model for identify several plant categories.

2.2 Dataset

Some reliable websites provide data for plant health monitoring including Plant Village, Kaggle and Google Web Scraper. According the instruction of professor I did collect the dataset from Computer Vision in Biosystem. The dataset includes more than 5K RGB photos of 12 different plant. To maintain the structure of the dataset directory the entire dataset I splited into training and validation sets in an 80/20 ratio. Here is the data set link (https://vision.eng.au.dk/plant-seedlings-dataset/)

2.3 CNN Structure

The Convolutional Neural Network (CNN) for plant categories identification input images of size 224x224x3 (height, width, and RGB channels), compatible with ResNet50's architecture.

Verification and Validation of Cyber Physical Systems

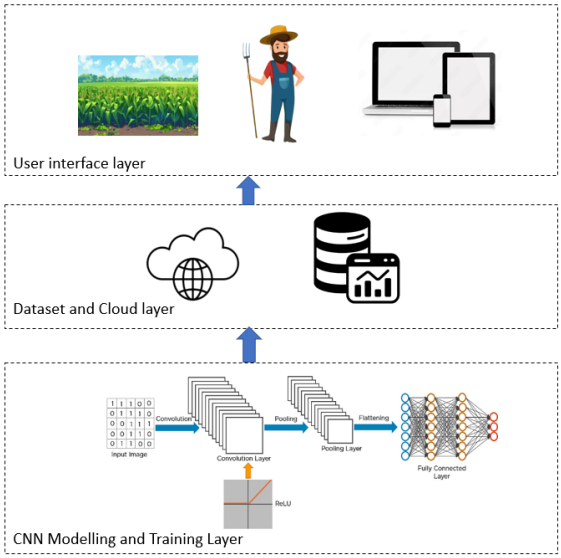


Fig. 2: System design

It utilizes ResNet50, a pre-trained deep neural network with 50 layers for feature extraction and classification tasks. ResNet50 uses residual blocks with skip connections to increase training efficiency and avoid vanishing gradients. The CNN consists of convolutional layers with increasing filters, progressively learning spatial and hierarchical features. There are total 48 convolutional layers are retained for feature extraction while one max-polling and one fully connected layer. The final fully connected layer is replaced to fit the specific task. The output layer, consisting of 12 neurons with softmax activation, predicts probabilities for each class. Data augmentation techniques, such as random horizontal flipping, resizing, and normalization were applied to improve generalization. The model contains approximately 23.5 million trainable parameters optimized with the Adam optimizer and Cross Entropy Loss for multi-class classification. This design enables efficient and accurate plant identification, leveraging the power of deep learning to classify images into 12 categories with high precision. In Figure 3 we can see the CNN model summary.

2.4 CNN implementation

For implementation purpose I did use several required library. To implement this CNN model I did use PyTroch. It is a popular open source neural network library written in Python. PyTroch usually run over the top layer engine and provide a easy platform for user to create and test deep learning models using Python. I used Sklearn for create a dataset including train and validation dataset from-directory with 80-20 ration respectively. It helps

Verification of Physical Systems

```
Total params: 23,532,620
Trainable params: 23,532,620
Non-trainable params: 0
----------------------------------------------
Input size (MB): 0.57
Forward/backward pass size (MB): 286.55
Params size (MB): 89.77
Estimated Total Size (MB): 376.89
```

Fig. 3: The summary of CNN

```
Epoch 23/30
----------
train Loss: 0.1223 Acc: 0.9580
val Loss: 0.2814 Acc: 0.9146
Epoch 24/30
----------
train Loss: 0.1053 Acc: 0.9609
val Loss: 0.2776 Acc: 0.9263
Epoch 25/30
----------
train Loss: 0.1014 Acc: 0.9695
val Loss: 0.2638 Acc: 0.9155
Epoch 26/30
----------
train Loss: 0.0755 Acc: 0.9715
val Loss: 0.4439 Acc: 0.8742
Epoch 27/30
----------
train Loss: 0.0994 Acc: 0.9666
val Loss: 0.2610 Acc: 0.9093
Epoch 28/30
----------
train Loss: 0.0970 Acc: 0.9675
val Loss: 0.2826 Acc: 0.9227
Epoch 29/30
----------
train Loss: 0.0766 Acc: 0.9745
val Loss: 0.2457 Acc: 0.9353
Epoch 30/30
----------
train Loss: 0.0686 Acc: 0.9772
val Loss: 0.3437 Acc: 0.9245
Best val Acc: 0.9353
```

Fig. 4: CNN Training Process

to avoid overfitting and quickly learning. I also used Troch vision which is useful for image pre-processing and augmentation. It is mandatory to keep same size of all input images before feeding them on the model. For train the model I did use 3 channels colored (RGB) images with resize dimension of 128 x 128 pixels. I did set 30 epochs. The training process of this model was done by using Intel CoreTM i7 CPU processor which has 16 GB RAM. The training process time around 10 hours to complete 30 at the time when I used my CPU but at university server it took around 30 minute. Every five epoch I did monitor about the progress of training process. This CNN model use softmax activation function in output layer cross-entropy as the loss function. I added 48 Convolutional Layers to extract more feature from images and increased number of the neuron for higher accuracy and avoid over

Verification and Validation for Cyber-Physical Systems

feeding. I did train my model more than five time then I get my desire accuracy and losses rate was become very low.

2.5 Experimental Evaluation

In the figure 5 we can see the accuracy results for training and validation set in individual line graph for per epoch. For the training and validation set we can see the accuracy increased according with the number of epochs in both cases. Initially we can see at the first epoch the training accuracy was only around 0.5 while validation accuracy was 0.55. After that at fifth epoch rapidly training accuracy was increased and reach at 0.86 beside that the validation accuracy also slightly increased to 0.78. Likewise training and validation both increased continually and the final training accuracy was 0.98 while validation accuracy was 0.93. That is really a great result for my plant health monitoring model.

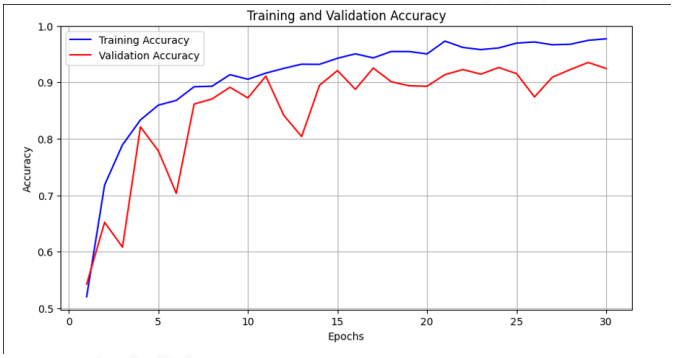


Fig. 5: Accuracy Results

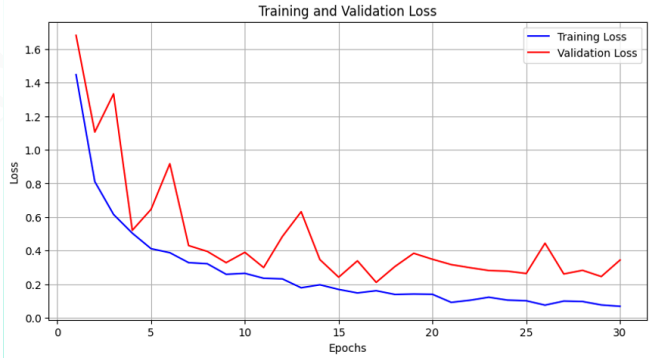


Fig. 6: Loss Results

In the figure 6 we can see Training and validation set loss results in individual line graph for per epoch. For the training and validation set we can see the loss decreased according

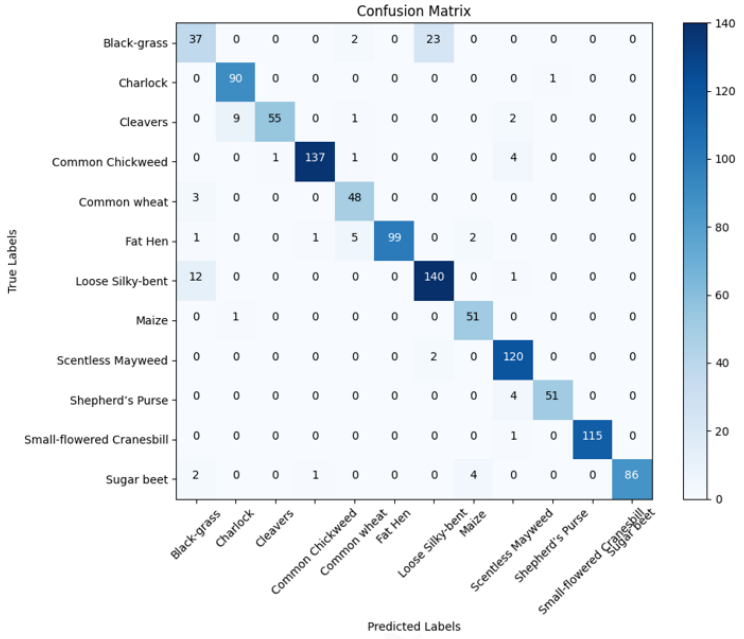


Fig. 7: Confusion matrix

with number of the epochs in both cases. Initially we can see at the first epoch the training loss was just above 1.4 while validation loss was 1.6 which is the maximum value in both cases. After that at fifth epoch rapidly training loss was decreased and reach at 0.4 beside that the validation loss also decreased to 0.6. After that the training and validation both loss decreased continually and the final training loss was only 0.1 while validation loss was only 0.3 which is really very less in both cases.

In the confusion matrix (Figure 7), we can see the accurate and wrong prediction data. The straight line demonstrate that the model can more precisely identify plant categories. Only two categories we can see the confusion in the model including Black grass and Loose Silky bent. It is also possible to solve by including more image on those categories.

2.6 Model Prediction Results

To evaluate the performance of the trained CNN model, I did build a small testing model where I load my pretrained model. During the testing phase, the saved model was loaded and predictions were made on the test set. After feeding image from test directory, we can see a good prediction of plant categories. I used Softmax activation vector on outer layer that's why it converts the model's raw outputs into probabilities and helps in multiclass classification by identifying the class with the highest probability. Here is some example of

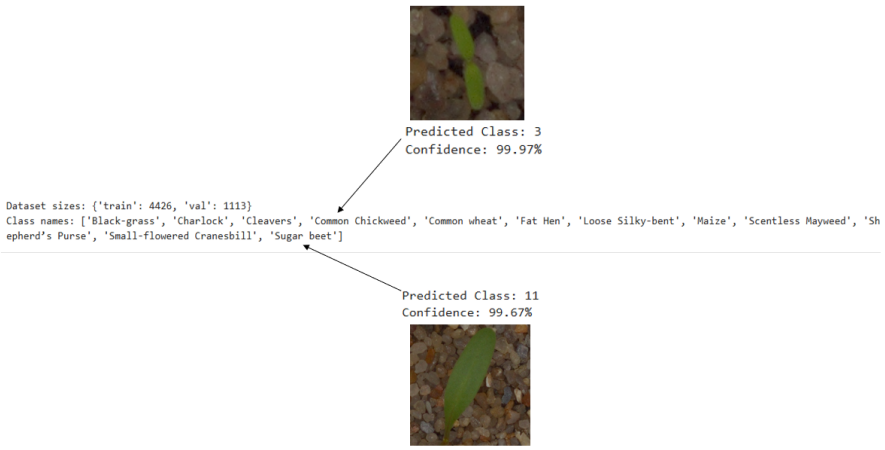


Fig. 8: Model Prediction Results

my test and prediction output. In the Figure 8 we can see, I did two experiment with two different images. In one case the model is predict with 99.97 percent confident that the plant is Common Chickweed while in another case 99.67 percent confident that the other plant is Sugar Weed. In both the prediction is totally correct with high accuracy. So we can see the model can identify 12 plant categories with high accuracy.

3 TAPAAL Model and Verification

We can use TAPAAL tools for design our precision farming concept. TAPAAL provide a good and easy user interface with several function, simulation and verification facilities. So according the system requirements I did use TAPAAL tools for designing and verification process. There are total three individual components including System_Controller, Drone_and Autonomous_Vehicles and Farmer_Control. Those component works independently and if need then make interaction among each other.

3.1 Use Case diagram

For design a TAPAAL model first of all we need to draw use case diagram for each individual components. In below I mentioned all relevant use case diagram for my system.

Verification and Validation of Cyber Physical Systems

3.1.1 Use case diagram for Precision Farming

In this Precision Farming diagram (figure 9), we can see System Controller as actor while three reactor such as Drone, Autonomous Vehicles and Farmer.

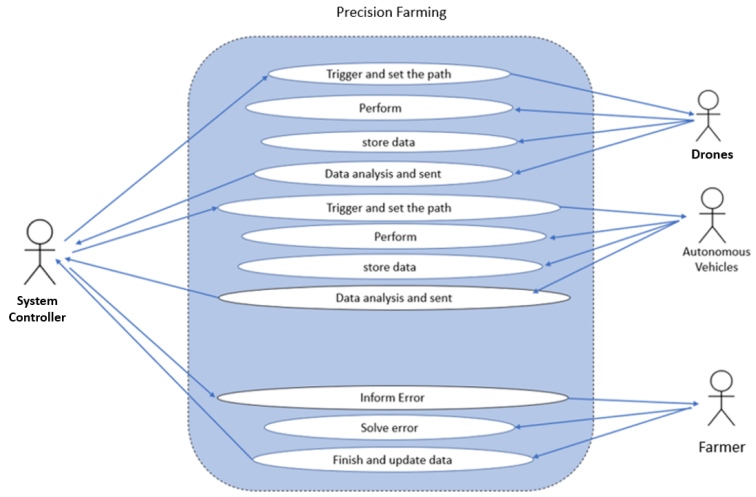


Fig. 9: Use case diagram for precision farming

- System Controller : It is responsible for control overall activities of the farm. We can see it trigger and set path for both Drones and Autonomous Vehicles. If any Physical Error identified then inform the farmer.
- Drones: The drones are reactor which always wait for instruction from the System Controller. After get instruction then is start it activities, store data, analysis data and sent to the Controller Unit.
- Autonomous Vehicles: It is also reactor and wait for instruction from System Controller. Whenever it get instruction then it start perform those tasks, store data, analysis data and sent those to the Controller Module.
- Farmer: Farmer is a reactor which only make interaction at the time of Error (physical error) handling. System Controller inform the farmer if any physical error identified by system then farmer come physically to solve the problem and then update the data to the System Controller.

Verification of Cyber Physical Systems

3.1.2 Use case diagram for Drones

This use case diagram shows the overall tasks and interaction of all drones with System Controller. I did use total five drone in five different rules.



Fig. 10: Use case diagram for Drones

- Survey Drone: After get instruction from System Controller then it will start to monitoring the farm and after finish monitoring then analysis the data and sent to the System Controller.
- Pest Control Drone: System Controller see the monitoring data and then it give the instruction to the Pest Control Drone for spray pesticide on infected area this time this drone activate and perform spray pesticide and after completion sent the data to System Controller.
- Watering Drone: System Controller give the instruction to irrigate on particular location this time the drone start perform irrigation and after finish sent the data to the System Controller.
- Weed Monitoring Drone: For identify weed on farm the System Controller sent instruction to weed monitoring drone then this drone start it activities and collect the weed data as well as sent to the System Controller.
- Harvesting Drone: This drone responsible for collecting harvesting data. System Controller give instruction to collect harvesting data from farm then the drone activated and start collecting harvesting data. After finish collecting then store and sent data to the System Controller.

Verification Copy ... sical Systems

3.1.3 Use case diagram for Autonomous Vehicles

In this Autonomous Section use case diagram, we can see a good interaction among Autonomous vehicles and System Controller. There are total two Autonomous vehicles working on my system.

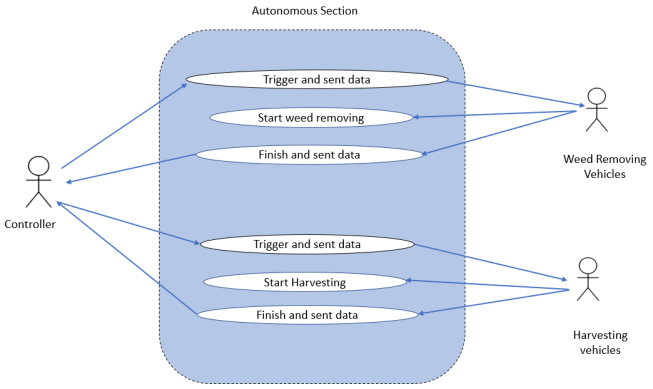


Fig. 11: Use case diagram for Autonomous Vehicles

- Weed Removing Vehicles: System controller give the instruction and data then weed removing vehicles start removing weed from those location and after finish sent data to the System Controller.
- Harvesting Vehicles: After get the data from harvesting drone the system controller give the instruction to the Harvesting Vehicles and then that vehicles start harvesting and after complete sent data to the System Controller.

3.1.4 Farmer Use Case

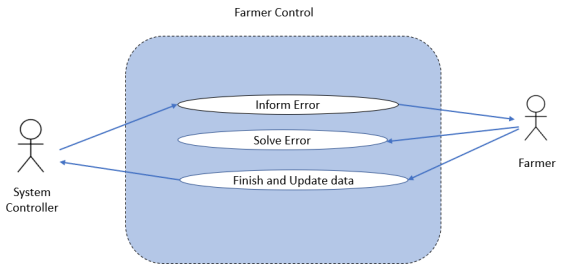


Fig. 12: Farmer Use case

In this use case diagram, we can see the interaction between System Controller and Farmer. If any physical error happened then System Controller is unable to solve that so it inform

Verification and Validation of Physical Systems

the farmer to solve the error. Farmer see the notification then solve the error and update the System Controller.

3.2 TAPAAL model for individual System

As I already mentioned that according the system analysis there are total three individual components for my system. According my system requirements and use case diagram, I did use Total Three components for managing overall system of precision farming including System_Controller, Drone_and_Autonomous_Vehicles as well as Farmer_Controll. All component can work independently and if interaction need that can also possible to interact each other's. In below I did describe those components.

3.2.1 System_Controller Interface

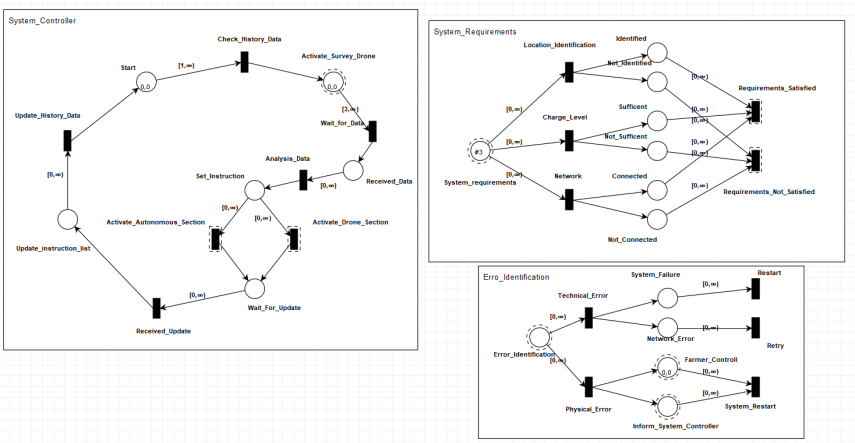


Fig. 13: System_Controller_Interface

The System_Controller executing all task and managing interaction among all components. First of all it Check_History_Data and then Activate_Survey_Drone to get the overall condition from the agricultural farm. After get survey data then it Analysis_Data and Set_instruction like need to Activate_Drone_Section or Activate_Autonomous_Section or sometime both. Then again Wait_For_Update from drone and autonomous section as well as after Received_Update it Update_instruction_list and Update_History_Data. There are two additional condition we can see here such as System_Requiremetns and Error_Identification. In System_Requirements section, usually check their Network connection, Charge_Level and Location_Identification for identify the drone and autonomous vehicles is ready or not ready for perform their tasks. If hey satisfied those System_Requirements then they are ready otherwise not ready. Another condition is Error_Identification which activated when

any error detected by the system. It helps to identify the types of error like physical_Error or Technical_Error and according the types of error it takes decision.

3.2.2 Drones_And_Autonomous_Vehicles Interface with Time Variants

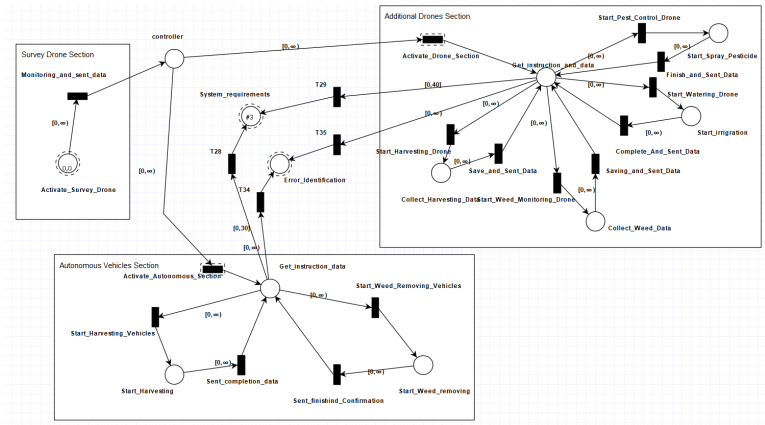


Fig. 14: Drone and Autonomous interface

According the system requirements and use case diagrams of Drones and Autonomous vehicles, I did design the TAPPAL model interface which is totally relevant and functional. First of all, controller analysis the survey data then take decision like which one need to activate Autonomous Vehicles or Drones section or both. For example, controller need to activate both section then it will send the coordinate data meanwhile it will check the System_Requirements. Since there is total four additional drone in the system that's why it will take 40s for verifying System_Requirements for Activate_Drone_Section and if Requirements_Satisfied then it will activate particular drone and that drone will be start and perform tasks as well as after complete task sent the data to controller. Here I did use total four additional drone including Pest Control, Watering, Weed Monitoring and Harvest Monitoring Drones. So the controller can activate all of them together or any particular drone as it's requirements.

Beside that for Activate_Autonomous_Section it will takes 20s for check System_Requirements and if Requirements_Satisfied then it will activate particular autonomous vehicles. Here I did use to autonomous vehicles for crops harvesting and Weed removing. So, the controller can select one or both according the survey data and it's requirements.

Verification of Cyber Physical Systems

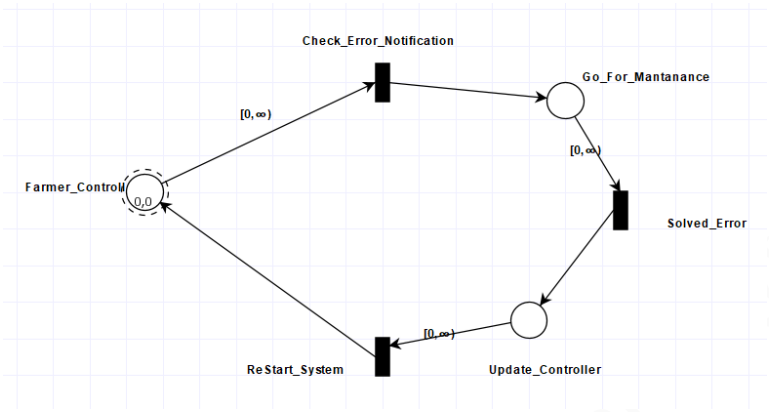


Fig. 15: Farmer interface

3.2.3 Farmer_Control Interface

In this interface we can see the Farmer_control section which get the notification about Physical_Error from controller and Error_Identification section then farmer come for maintenance the physical error and Solved_Error then Update_Contoller and restart the system.

3.3 TAPAAL Verification

In verification section I did total four verification including System_Controller_Reachability, Farmer_Controller_Reachability, DeadLock, Controller_Model_liveness. Those four verifications were successfully done and all property was satisfied. In below I discussed all verification process individually.

3.3.1 System_Controller_Reachability

This Verification process for the System_Controller_Reachability analysis to ensure that all critical system states are achievable during operation. The formula: (EF System_Controller.Update_instruction_list >= 1) indicates that it is possible for the System_Controller to reach a state where at least one instruction has been updated in the instruction list. This is a proof that System_Controller can update the instruction list according the requirement and control the system.

Verification and Validation of Cyber Physical Systems

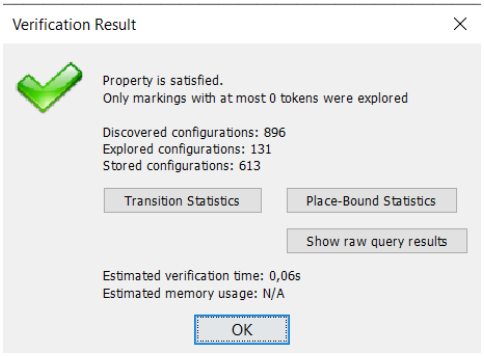


Fig. 16: system_Controller_Reachability

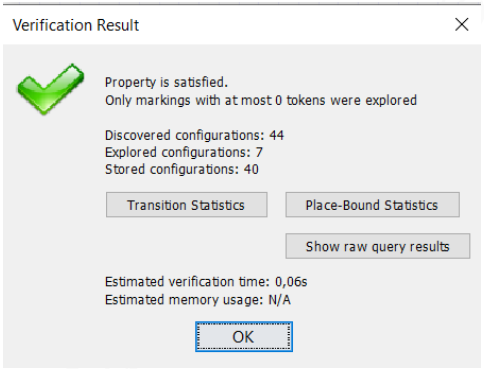


Fig. 17: Farmer_Controller_Reachability

3.3.2 Farmer_Controller_Reachability

This TAPAAL verification process for the Farmer and Controller reachability analysis to ensure manual instruction is possible when needed. The formula: (EF Farmer_Control.Update_Controller >= 1) ensures that the system can reach a state where the Farmer_Control updates the system manually. This guarantees that in cases of physical errors, the farmer can maintenance manually and solve the error.

3.3.3 Deadlock

The TAPAAL verification process for deadlock verification uses the formula: (AG !(deadlock)) This ensures that the system is free from deadlocks. It guarantees the continuous functionality of the system, ensuring tasks are always processed without any encounter.

Verification and Validation of Cyber-Physical Systems

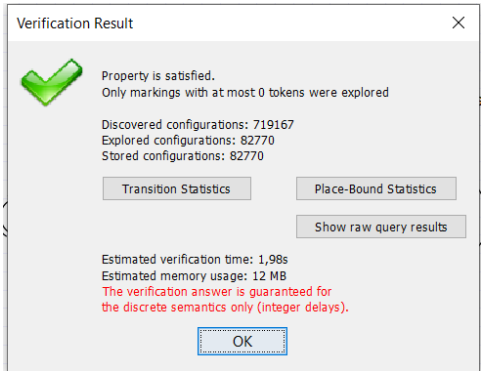


Fig. 18: Deadlock

3.3.4 Controller_model_liveness

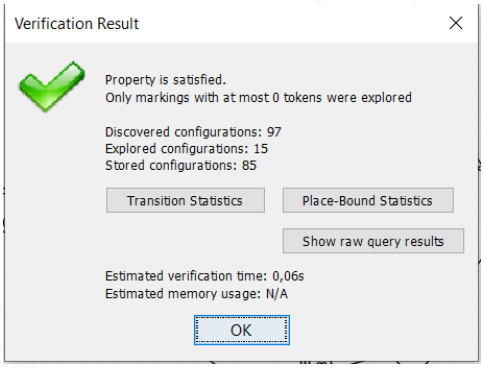


Fig. 19: Controller_model_liveness

In this verification process for liveness in the Controller uses the formula: (EF !(System_Controller.Network_Error = 0 and System_Controller.Received_Data = 0)) This ensures that the system remains älive"by verifying that it is always possible to reach a state. It confirms that the system continues to operate effectively without any encounter.

4 Competition

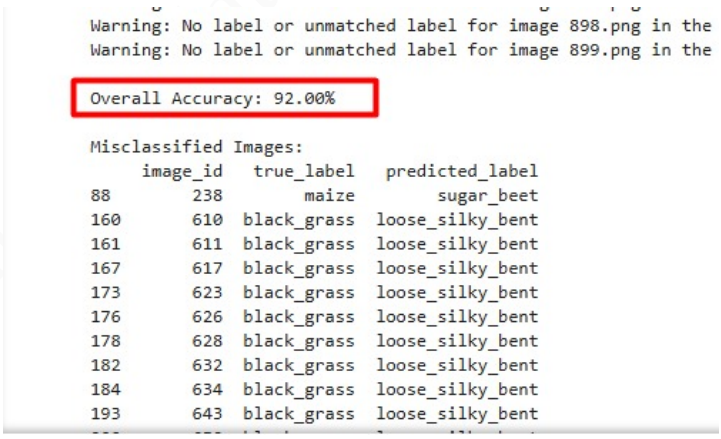
There was total four team among them our professor tried to organized a competition for identify plant categories. There is many perimeter to choose the winner on competition like the overall accuracy of plant categories identification among the dataset of own and others group.

4.1 Preparation for competition

First of all, I did fulfill the basic requirements for joining the competition like make a dataset with 900 image with CSV file and a trained .pkl model. Then I just made a tasting model which can give me overall accuracy of my model for the dataset and CSV file. For that I just called my pretrained model and by importing some library I was able to get the overall accuracy. So that time I was ready to participate on competition and then I start to develop my dataset and make it more complex for others group.

4.2 Approach To Win the Competition

First of all, I planned to train my model with more data then my model can learn better. So, I did copy all images from Validation folder to Train folder then again I copied all image from individual categories of training directory and pasted on the respective categories for twice. Apart from that I did download new images from Kaggle website and add that on my training directory. From my confusion matrix I noticed that two categories like Black grass and Loose Silky bent predict wrong for several time on my previous training process so I did download more image on those two categories and included that on those directories. So this time I had enough image for train the model and I did get desire accuracy after run 30 epochs. For creating folder with 900 image I did apply my best trick to win the competition like I took 800 images from Black grass and Loose Silky categories and 100 images from other 10 categories.



```
Warning: No label or unmatched label for image 898.png in the
Warning: No label or unmatched label for image 899.png in the

Overall Accuracy: 92.00%

Misclassified Images:
     image_id   true_label   predicted_label
88        238        maize        sugar_beet
160       610  black_grass  loose_silky_bent
161       611  black_grass  loose_silky_bent
167       617  black_grass  loose_silky_bent
173       623  black_grass  loose_silky_bent
176       626  black_grass  loose_silky_bent
178       628  black_grass  loose_silky_bent
182       632  black_grass  loose_silky_bent
184       634  black_grass  loose_silky_bent
193       643  black_grass  loose_silky_bent
```

Fig. 20: Overall Results for 900 images

According that I build a CSV file also. For checking performance on those 900 image I did run my testing model I get 92 percent accuracy for my dataset and I think I model will be achieve such good accuracy for other groups dataset but since my dataset has more heard categories images that's why I will win the competition.

5 Conclusion

We did explore Precision Farming in this paper which combines advanced technologies such as deep learning and formal verification to solve challenges in traditional agriculture. By Trained a deep learning model trained with over 5,000 images with high-accuracy identification of plant categories and weeds is achieved which possible to use for optimizing the use of pesticides and fertilizers as well as minimizing waste. The TAPAAL-verified model integrates drones and autonomous vehicles controlled by system controller to automate essential tasks like monitoring, irrigation, pest control, and harvesting. This combination of trained deep learning model and TAPAAL verified model ensures efficiency, scalability, and sustainability in modern agriculture. Finally, we can see this study demonstrates how Precision Farming can be a good solutions for farming practices, ensuring food security and environmental sustainability for a growing global population.

6 Project Management with GitHub

Actually, GitHub is a really good tools for project management. But unfortunately, in Group A, I am only member who participate on the course until end because other two member was unable to finished their seminar paper on time that's why they quit the course. So, on the GitHub repository all contribution is only done by me. I did upload all lab tasks on that repository and here you can see the graph of my activities.



Fig. 21: GitHub Contribution

7 Declaration of Originality

I, Md Akram, herewith declare that I have composed the present paper and work by myself and without the use of any other than the cited sources and aids. Sentences or parts of sentences quoted literally are marked as such; other references with regard to the statement and scope are indicated by full details of the publications concerned. The paper and work in the same or similar form have not been submitted to any examination body and have not been published. This paper was not yet, even in part, used in another examination or as a course performance. I agree that my work may be checked by a plagiarism checker.

Verification and Validation of Cyber-Physical Systems

24-01-2025, Lippstadt MD Akram

Date&Place - Full Name

Bibliography

[AR21] Ahmed, A.A.; Reddy, G.H.: A Mobile-Based System for Detecting Plant Leaf Diseases Using Deep Learning. AgriEngineering, 3:478–493, 2021.

[Kr24] Krishna, Ujjwal; Bharadwaj, Sreepada; Kaswan, Vinit; Kumar, Anuraj; Kaur, Gursimran; Rana, Pooja: Agri Watch: Precision Plant Health Monitoring using Deep Learning. E3S Web of Conferences, 556:01028, 2024.

[Ne21] Network, Crop Protection: Crop Protection Network, 2021. Available online: `https://cropprotectionnetwork.org/` (accessed on 25 June 2021).

[Za24] Zain, Muhammad; Ma, Haijiao; Ur Rahman, Shafeeq; Nuruzzaman, Md.; Chaudhary, Sadaf; Azeem, Imran; Mehmood, Faisal; Duan, Aiwang; Sun, Chengming: Nanotechnology in precision agriculture: Advancing towards sustainable crop production. Plant Physiology and Biochemistry, 206:108244, 2024.

Verification and Validation of Cyber-Physical Systems